ADR4AI: LLM-Based Architectural Pattern Selection and ADR Generation for AI-Enabled Systems

Anonymous Authors

Working draft: Tasks 1.1–1.6 only. The ADR-generation description is pending human review. This document is not a complete paper or a submission-ready manuscript.

I. Methodology

A. Research Questions and Study Scope

We organize the study around three research questions (Table I). The computational protocol treats a user story and its acceptance criteria as a requirement case, with the system purpose providing context. It assesses each catalog pattern against each case independently. The assessment distinguishes applicability, non-applicability, and insufficient information; a positive classification requires a supporting user-story or acceptance-criterion reference. These classifications are model outputs, not independently established ground truth.

RQ1 characterizes the model's classification outcomes rather than measuring their correctness. RQ2 compares the patterns classified as applicable with a separate retrieval baseline. This baseline retrieves catalog candidates and reranks them to a final Top-5 without making an LLM-based applicability judgment. Overlap is therefore a descriptive relationship between two output sets, not evidence of either approach's precision, recall, or superiority.

ADR generation is a separate stage: each pair classified as applicable supplies the requirement case, selected pattern, and applicability evidence for one generated ADR. Generation completeness and stored provenance establish the availability and traceability of these records, not their architectural quality. RQ3 examines expert assessments of these selected pairs and their corresponding ADRs, rather than adding another automatic applicability classification.

The prepared questionnaire measures decision adequacy for the stated requirements (Q1), decision grounding (Q2), presentation of relevant consequences and trade-offs (Q3), perceived usefulness as a decision record (Q4), and pattern applicability (Q5). These are five separate expert-assessed dimensions; they are not a validated composite quality score. Optional comments provide context for the ratings. Neither implementation feasibility, cost savings, novelty, nor actual improvement in architectural practice is directly measured by these items.

Experts evaluate cases for a system they know, and only pairs selected by the LLM are presented. Consequently, RQ3 can characterize expert judgments of the presented recommendations but cannot identify missed applicable patterns or estimate recall over the full set of assessed pairs. Expert agreement with a recommendation does not by itself establish implementation success or objective correctness. Repeated ratings from an expert and multiple cases from a system must not be treated as independent replications when interpreting the results. Conclusions remain bounded to the sampled requirements, catalog, generated records, and participating experts.

B. Industrial Requirement Sample

The study uses a bounded industrial sample comprising 4 systems, 12 user stories (3 per system), and 35 acceptance criteria (Table II). The supplied data-scope document defines this as a minimal, delimited sample for studying architectural-pattern selection and decision documentation for AI-enabled systems. It is not a representative sample of industrial requirements or AI-enabled systems. The available artifacts do not establish a probabilistic sampling procedure; we therefore do not interpret these counts as population estimates.

Each requirement case consists of the system purpose, one stakeholder user story, and all of that story's acceptance criteria. The source CSV stores one acceptance criterion per row, repeating the corresponding purpose and story. Grouping by the stable user-story identifier reconstructs the case without rewriting the original Portuguese requirement text or changing criterion order. The criterion-row count must not be confused with the number of requirement cases.

System purpose provides contextual information, but is not sufficient on its own to establish pattern applicability. Positive evidence must be grounded in the user story and/or its acceptance criteria. The model is not authorized to infer unstated technologies, requirements, or constraints from the purpose.

The source uses generic names `System_01`–`System_04`; the paper uses the corresponding stable identifiers `SYS01`–`SYS04`. Story and criterion identifiers preserve traceability within each system without exposing real system names. The organization is not named in this review draft. According to the supplied scope document, the study input is restricted to purposes, identifiers, user stories, and acceptance criteria; it does not include operational databases, personal data, credentials, customer records, or production content. This delimitation does not itself establish participant consent, ethics approval, or permission for unrestricted public release.

TABLE I
RESEARCH QUESTIONS, UNITS OF ANALYSIS, AND EVIDENCE MAPPING.

ID	Research question	Unit, evidence, and intended analysis
RQ1	How does the LLM classify architectural-pattern applicability across the requirement cases, and which patterns does it identify as applicable?	Unit: one user-story–pattern pair. Evidence: the official exhaustive assessment log, original requirements, and pattern catalog. Analysis: distributions of the three classifications and applicable-pattern sets by story and system, retaining zero-positive stories.
RQ2	To what extent do the patterns classified as applicable by the LLM overlap with the frozen RAG baseline's final Top-5 candidates for the same requirement cases?	Unit: one LLM-positive user-story–pattern pair, grouped by story. Evidence: official applicability assessments and the frozen final Top-5 rankings. Analysis: membership-intersection counts by story and overall, with the LLM-positive set as an explicit denominator; no applicability-accuracy metric.
RQ3	How do experts assess the applicability of the LLM-selected patterns and the adequacy, grounding, consequences, and perceived usefulness of the corresponding generated ADRs?	Unit: one expert's rating of one selected pair and its ADR on one questionnaire item. Evidence: consolidated expert responses to Q1–Q5 and optional comments for the prepared cases. Analysis: item-specific ordinal distributions and case-level summaries with response counts; comments contextualize ratings. Aggregation and inclusion rules must be specified before consolidated analysis.

TABLE II
THE BOUNDED INDUSTRIAL REQUIREMENT SAMPLE.

System ID	User stories	Acceptance criteria
SYS01	3	6
SYS02	3	11
SYS03	3	7
SYS04	3	11
Total	12	35

C. Architectural Pattern Catalog

We use 70 pattern records from the repository associated with Heiland et al.'s multivocal literature review of design patterns for AI-based systems [1], [2]. For provenance verification, we compared the local files with the public repository at revision `9ff3e61` (the full revision is recorded in the artifact audit). All 70 records matched the corresponding repository files byte-for-byte, as checked using Git blob hashes. This identifies the source snapshot of the catalog; it does not establish the effectiveness of the individual patterns.

Each pattern is stored as one JSON record with a stable three-digit identifier, from `001` through `070`. The applicability prompt supplies exactly five catalog fields: *ID*, identifying the record; *name*, naming the pattern; *motivation*, describing the problem or need; *solution*, describing its proposed response; and *consequences*, describing resulting effects and trade-offs. Identifiers remain strings, preserving leading zeros and enabling joins with the assessment and ADR artifacts. Source text is preserved rather than normalized or rewritten.

The records also contain aliases, examples, related-pattern identifiers, categories, and resource identifiers. These additional fields are not supplied to the applicability model. Resource identifiers provide source pointers within the repository, not independently verified bibliographic descriptions in the local catalog. We therefore attribute the collection to the verified repository and its associated publication rather than infer individual source citations from numeric resource codes.

All five input fields are present as strings in every record. Explicitly empty motivation or consequences strings are retained as empty inputs, without inventing replacement descriptions. A missing field or non-string value is an input-validation failure, not an invitation to repair the source. This preservation policy keeps the assessed catalog traceable while acknowledging uneven descriptive completeness.

The catalog includes general architectural patterns, such as Client–Server, alongside patterns concerning machine-learning safeguards and model serving. We use the records as a bounded candidate set, not as equally mature, mutually independent, or universally applicable solutions. Catalog inclusion alone does not establish applicability to a requirement case; that relationship is assessed separately.

D. Exhaustive Pattern-Applicability Assessment

The experiment crosses all 12 requirement cases with all 70 catalog patterns, yielding $12 \times 70 = 840$ planned pairwise assessments. Each request contains one complete requirement case and one pattern, with no prior assessment or conversation history supplied. This exhaustive design does not retrieve or prefilter candidates. Separate requests do not imply statistical independence: assessments share stories, systems, catalog entries, and a model. The resulting classifications are model predictions, not expert-provided ground-truth labels.

a) Model and structured response: The recorded model identifier is `gpt-6-sol`, with reasoning effort `high`. The implementation uses the Responses API with strict JSON-schema output named `pattern_assessment`. The schema requires exactly three fields: `classification` (one of the three labels below), `requirement_refs` (an array of strings), and `rationale` (a string); additional properties are disallowed. Each stored result also includes the story ID, pattern ID, model identifier, and reasoning effort. No temperature or seed is set in the request, and the recorded model identifier does not pin a dated model snapshot.

b) Decision and grounding rules: The instructions distinguish:

- `applicable`: the story and/or its acceptance criteria provide positive evidence of a need, problem, constraint, or quality concern addressed by the pattern.
- `not_applicable`: those requirements provide evidence that the pattern does not fit the stated need, problem, constraint, or context.
- `insufficient_information`: the story and criteria do not provide enough evidence to establish either applicability or non-applicability.

System purpose supplies context but cannot alone justify applicability. The prompt prohibits assuming unstated requirements, technologies, constraints, or system characteristics. Absence of contradiction, possible usefulness, or the possibility of implementing, testing, or improving a requirement using a pattern is not sufficient positive evidence. Assessment concerns underlying architectural intent rather than literal terminology. An applicable response must cite at least one identifier belonging to the current story or one of its criteria, and briefly explain the evidence–pattern relationship; it must not propose an architecture, implementation, technology, or new requirement. These are prompting and reference-validation constraints, not proof that a model's rationale is semantically correct.

c) Execution, retries, and validation: Execution is sequential, in CSV story order and ascending pattern-ID order. SDK retries are disabled. The runner permits at most 3 attempts per pair, revisiting unresolved pairs in up to 3 ordered passes rather than immediately retrying each failed request. Before a request, it atomically persists an attempt reservation, so interrupted requests consume the attempt budget. A request failure updates that entry with its error type and a whitespace-normalized message, truncated to 300 characters with the configured API key redacted. Successful validation removes the pair from this unresolved-failure state.

The response must have completed, contain no refusal, and provide parseable structured output. Local validation checks the exact fields, label, reference array and identifier membership, a nonempty reference array for `applicable`, and a string-valued rationale. These checks do not require a nonempty rationale string or test the factual entailment of cited text. Only validated results are appended to the official log, flushed, and synced to disk; storage failures stop execution rather than risk duplicate writes. A nonblocking file lock prevents concurrent official runners.

On resume, the runner validates existing JSONL records, model configuration, pair uniqueness, and failure-state attempt counts. Malformed records, unknown identifiers, and interrupted writes lacking a final newline stop the run rather than being repaired or discarded. Completed pairs are skipped; unresolved attempt counts are retained, and exhausted pairs are not retried. The final on-disk check requires exactly all 840 expected pairs and 70 records per story before exporting story-specific applicable-pattern selections.

d) Official dataset and replication artifact: The official dataset is the top-level `assessments.jsonl` under the pattern-selection experiment's `outputs` directory, accompanied by `failures.jsonl`. Separate `pilot_SYS01-US01` directories (including the `_v1` and `_v2` variants) and `test_assessment.json` and `test_assessment_v2.json` are excluded, not merged with official records. An offline audit verifies complete official coverage and records input/source hashes, the exact instructions, JSON schema, and input template. The template presents system purpose, identified story and criteria, followed by the five pattern fields in Section I-C; the full instructions remain in the artifact rather than this space-bounded description.

The frozen log contains 840 validated pair outcomes and an empty final unresolved-failure file. This establishes completion, not that no retries occurred: successful records omit attempt counts, and successful retries remove failure-state entries. Thus, 840 describes the planned first-pass requests and retained outcomes, not an independently verified total number of API attempts. We do not claim exact stochastic reproduction from the stored configuration.

E. Retrieval and Reranking Baseline

We compare the LLM-positive pairs with a separate retrieval/reranking baseline, called the RAG baseline in the artifacts. Despite that name, this pipeline performs no generative step: it uses no external LLM to classify applicability or generate ADRs. Its final rankings are candidate recommendations, not validated applicability judgments.

a) Query and catalog representation: The baseline groups the English requirement CSV by story ID, retaining the CSV encounter order. One query per story concatenates labeled system purpose, the user story preceded by its bracketed identifier, and all acceptance criteria, each preceded by its bracketed identifier. Identifiers are preserved in the query and output for traceability, not substituted for requirement text. An offline audit verifies that the story/criterion identifiers and their ownership match the original sample. The baseline uses the English requirement copy, whereas the applicability experiment uses the original Portuguese text; this is not a language-controlled comparison, and identifier alignment does not itself establish translation equivalence.

The retrieval corpus contains one document per catalog pattern. It includes the pattern ID and nonblank source values for name, aliases, motivation, solution, consequences, examples, categories, related patterns, and resources; list entries are joined with semicolons, omitting blank entries. These catalog records match the original pattern fields, but the retrieval representation is richer than the five-field applicability input. The Chroma collection is versioned by a fingerprint of catalog content, text-format version, and embedding-model identifier. Index compatibility checks verify complete IDs, document text, and metadata before reuse, rebuilding incompatible contents.

b) Vector candidates and final ranking: The embedding model is `BAAI/bge-small-en-v1.5`. Chroma vector similarity search retrieves 15 candidate documents per query (vector Top-15), using the library's default similarity configuration without overriding the distance metric. The cross-encoder `BAAI/bge-reranker-base` scores each query–

candidate pair. Candidates are sorted by descending reranker score, with vector rank breaking ties, to retain the final Top-5. The implementation requires exactly 15 vector candidates and one finite reranker score per candidate. Thus, Top-5 is a reranked subset of Top-15, not the first five vector results. The stored output preserves the vector ordering and the final reranked ordering separately.

c) Candidate-overlap measure: Let S be the set of sampled stories, A_s the set of pattern IDs classified as `applicable` by the LLM for story s, and R_s the set of IDs in the frozen final Top-5 for that same story. For each LLM-positive pair (s, p), define $m(s, p) = 1$ when $p \in R_s$, and $m(s, p) = 0$ otherwise. The per-story overlap count is $o_s = |A_s \cap R_s|$. The pooled LLM-positive-set overlap proportion is

$$O = \frac{\sum_{s \in S} |A_s \cap R_s|}{\sum_{s \in S} |A_s|}, \qquad \sum_{s \in S} |A_s| > 0. \tag{1}$$

This weights positive pairs, not stories equally; it is not an average of story-level percentages. A zero-positive story remains in the sample with $o_s = 0$, but contributes neither a positive pair nor a per-story percentage. If no story has a positive pair, O is undefined, not zero. Only final Top-5 membership enters this measure: vector Top-15 membership, ranking position, and reranker scores do not. Since A_s consists of model predictions rather than independently established truth, O measures candidate overlap, not precision, recall, accuracy, ground-truth applicability, or superiority of either approach. A nonmatch does not demonstrate a missed applicable pattern.

d) Frozen source and reproducibility limits: The comparison uses `rag_baseline.csv` in the RAG experiment's `outputs` directory, not the separate `rag_baseline_reproduced.csv`. An offline audit validates one frozen row per story, unique known IDs, ID/name alignment, and final Top-5 inclusion in vector Top-15, without recomputing rankings. The package records fixed model identifiers and pins direct dependencies and numerical runtimes, but does not pin downloaded model revisions or fully lock transitive dependencies. Rebuilt approximate vector indices and numerical hardware differences may change ranks. Catalog/text compatibility checks do not eliminate these sources of variation. No embedding models, reranker, or index were executed for this manuscript audit, and exact end-to-end reproduction is not established by these structural checks.

F. Architecture Decision Record Generation

Figure 1 summarizes the pipeline and its separate retrieval comparison branch. ADR generation consumes the 36 unique (`user_story_id`, `pattern_id`) pairs classified as `applicable`. It produces one proposed record per pair, not a jointly optimized architecture or an expert-approved decision. The accepted protocol pilot is included in the 36 records.

a) Inputs and prompting: Each request supplies the system purpose, identified story and all criteria; the selected pattern's ID, name, motivation, solution, and consequences; and the prior applicability assessment's evidence references, corresponding requirement text, and rationale. RAG rankings and overlap flags are excluded. The prompt permits legitimate empty catalog descriptions and limits generation to one contextualized architectural decision. It prohibits inventing unsupported requirements, technologies, components, protocols, or implementation details, or merely repeating that the system will adopt the named pattern. System purpose is context, while the story and criteria supply requirements. The proposed decision must instantiate the pattern's architectural intent; consequences must follow from the supplied information rather than portray the decision as universally beneficial.

The Nygard-inspired structure [3] comprises a short title, neutral context describing the relevant forces, an active-voice decision phrased as "We will ...", and relevant positive, negative, and neutral consequences. These are prompting rules, not automatically verified semantic properties. One synthetic recruitment/compliance example demonstrates the transformation using catalog Pattern `007`, which is outside the selected ADR-generation pairs. The example is instructional, not an empirical case. The generated ADR must not mention the model, experiment, applicability process, or evidence identifiers. Source references and rationale remain in the surrounding research artifacts for traceability.

b) Configuration and output validation: Generation uses `gpt-6-sol` with reasoning effort `high` and one independent Responses API request per ADR, without prior generated ADRs in its input. The client disables SDK retries and sets a 600-second timeout. No temperature or seed is supplied, and the model identifier is not a dated snapshot. Strict JSON-schema output, named `adr_pilot` for both pilot and full execution, requires exactly four string-valued fields: title, context, decision, and consequences, disallowing additional properties. Local validation rejects incomplete responses, refusals, duplicate JSON keys, incorrect field sets, and empty or non-string fields. The application then adds the fixed status `Proposed`; the model neither selects nor outputs that status. Neither schema compliance nor a Proposed status establishes architectural adequacy or stakeholder acceptance.

Each completed JSON artifact stores its pair identifiers, model and reasoning effort, response ID, `api_calls=1`, source-file path and SHA-256, rendered-prompt SHA-256, strict-output validation flags, and the five ADR fields. The source hash identifies the pre-ADR selected-pair CSV bytes, now retained in the backup: the current selected-pair CSV additionally contains an ADR column and therefore has a different file hash. The offline compiler validates pair coverage, metadata, source/prompt hashes, and fields before joining outputs by pair IDs. ADR text is retained as generated, not normalized, translated, or architecturally corrected.

c) Pilot, execution, and authorized retry: The research artifacts document approval of the `SYS01-US01`–Pattern `001` pilot. Full execution reuses that record without regenerating it and processes only pending pairs sequentially. It validates existing outputs and refuses duplicate or overwritten completed records. Persistent attempt claims prevent silent retries after

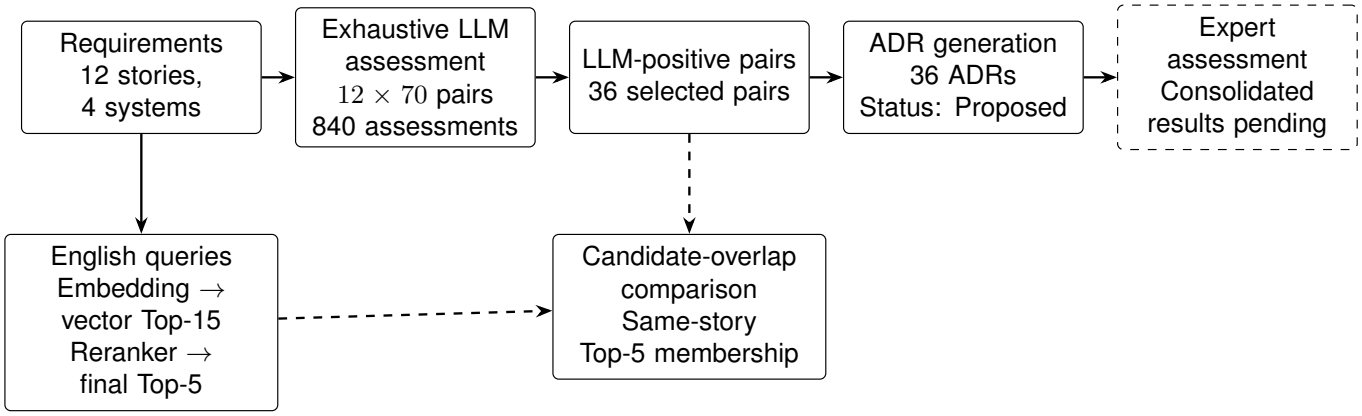


Fig. 1. ADR4AI pipeline. The retrieval branch supplies candidate rankings for comparison, not ADR-generation input. Dashed comparison arrows denote final-Top-5 membership; the dashed expert stage has no consolidated results in this draft.

interruptions; failure records retain the error, and any failure stops execution. Publishing full-run artifacts uses exclusive atomic creation rather than replacement.

For `SYS04-US03`–Pattern `003`, the archived failure reports "Connection error." A second attempt explicitly records manual retry authorization and links to the original claim. Both claims and the failure remain execution history, separate from the one completed ADR for that pair. Because a connection failure may precede dispatch, this failed attempt cannot be equated with an additional completed model response. The pilot plus final outputs cover exactly 36 unique selected pairs; attempt metadata is not counted as additional ADRs. Pilot approval concerned the generation protocol, not the future experts' judgments. Consolidated expert assessment is still pending in this draft.

REFERENCES

[1] L. Heiland, M. Hauser, and J. Bogner, "Design patterns for AI-based systems: A multivocal literature review and pattern repository," in *2023 IEEE/ACM 2nd International Conference on AI Engineering – Software Engineering for AI (CAIN)*. IEEE, 2023, pp. 184–196.

[2] SWE4AI, "Design patterns for AI-based systems: A web-based repository," [Online]. Available: https://swe4ai.github.io/ai-patterns/, 2023, repository snapshot 9ff3e61; accessed October 12, 2026.

[3] M. Nygard, "Documenting architecture decisions," [Online]. Available: https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions, 2011, November 15; accessed October 12, 2026.